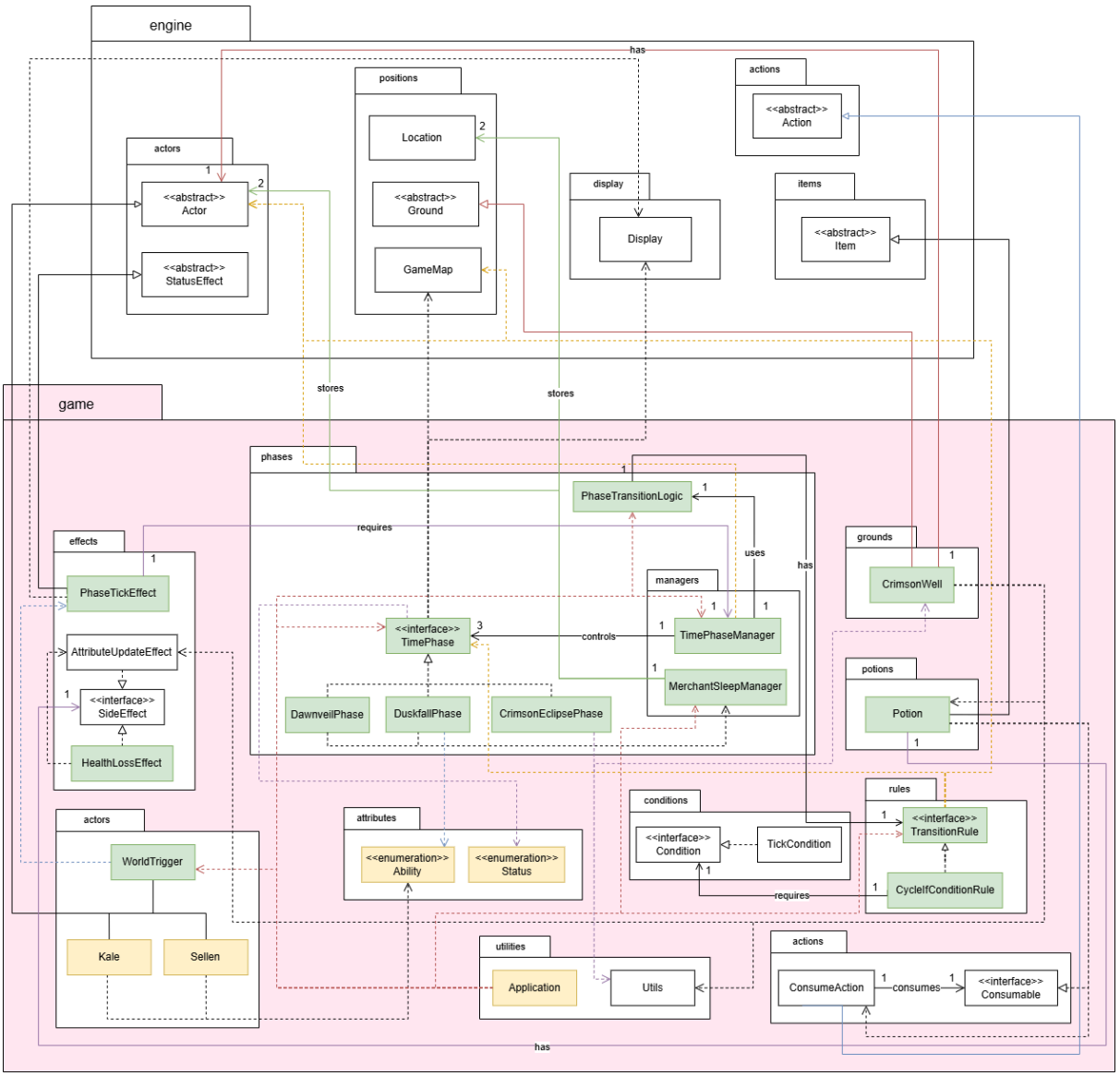
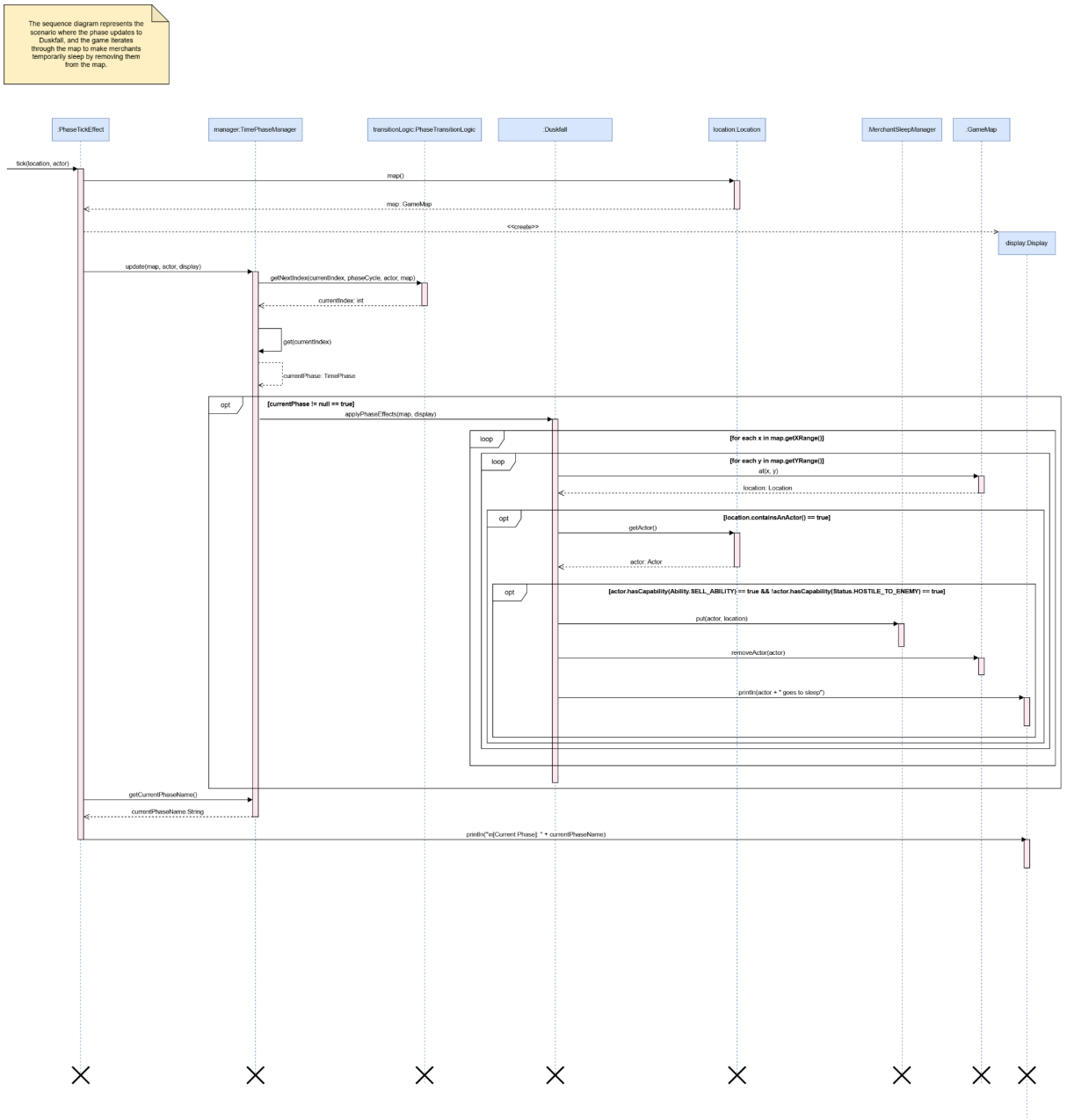
Req 3 - Creative Mode

UML Diagram:



Existing Class	White
Modified Class	Yellow
New Class	Green

Sequence Diagram:



Design 1

Approach 1:

Create a TimePhase interface to represent each world phase such as DawnveilPhase, DuskfallPhase and CrimsonEclipsePhase. Each phase implements this interface to apply its own map-wide effects through applyPhaseEffects() and expose its name via getPhaseName().

Pros	Cons
Each phase class defines its own effects, keeping the logic self-contained and easy to reason about	Common logic needs to be repeated in each class if multiple phases share similar behaviour
Supports polymorphism, allowing TimePhaseManager to handle all phases through the same interface without knowing specific types	
New phases can be added by simply implementing the interface, without modifying the manager or existing phases	

Approach 2:

Instead of using an interface, define a base abstract class for all phases and implement shared logic in the superclass.

Pros	Cons
Allows sharing of common logic across phases	Forcing unrelated phases to share inherited behaviour
	Tightly couples all phases to a single hierarchy, reducing flexibility

Future extensibility (Approach 1):
If a new phase like BloodmoonPhase is added in the future, it can be introduced by simply implementing the TimePhase interface and added to the manager's phase list without modifying any existing code.

Design 2

Approach 1:

Create a MerchantSleepManager class to temporarily store merchants removed from the map during the Duskfall phase and re-add them during Dawnveil.

Pros	Cons
Keeps all the sleep-related logic in one place instead of spreading it across different phases	Need to pass the manager into each phase that uses it, which adds a bit more setup
Can be reused in any phase that needs to manage sleeping or waking merchants	
Avoids using static variables, which helps hide internal data and keeps it safe from unwanted changes	

Approach 2:

Move merchant removal and restoration logic directly into each phase (eg. Duskfall and Dawnveil) instead of using a central manager.

Pros	Cons
Reduces the need to pass a shared object into phases	Duplicates merchant handling logic across phases, violating DRY
	Requires use of static or global variables to persist data between phases, increasing coupling and hidden dependencies

Future extensibility (Approach 1):
If another phase like BloodmoonPhase needs to put merchants to sleep or wake them up, the current design allows it to reuse the same MerchantSleepManager without duplicating code or modifying existing phases.

Design 3

Create a TimePhaseManager class to manage the current world phase and handle transitions. It stores a fixed list of phases and uses a PhaseTransitionLogic to determine when to change to the next phase. Each turn, it checks whether the current index should be updated based on the logic, and then applies the active phase's effects.

Pros	Cons
Keeps all phase transition logic in one place, reducing duplication across phases	Requires external setup of PhaseTransitionLogic and rules, making initial setup slightly more involved
Separates phase logic from transition logic, keeping responsibilities focused	Assumes that phase list ordering matches intended transition sequence, which may need careful tracking
Works with any number of phases and can cycle or jump conditionally	

Future extensibility:
If a new phase like BloodmoonPhase is introduced later and requires special conditions to trigger (e.g. standing on multiple cursed tiles), it can be added to the list and handled by writing a new rule. This avoids modifying TimePhaseManager, since all logic is injected via PhaseTransitionLogic.

Design 4

Create a TransitionRule interface to define how phase switching should happen. Implement CycleIfConditionRule to move to the next phase in a fixed cycle when a given Condition (Tick Condition) is satisfied.

Pros	Cons
Keeps rule logic decoupled from the manager	Rules rely on index logic, so phase order must match intent
Reusable with any phase list and condition due to abstraction	

New transition behaviours can be added without changing existing code	

Future extensibility:
If more complex rules are needed later (e.g. jump directly to a specific phase, skip phases), they can be added by implementing the same TransitionRule interface. No changes are needed to the manager or other rules.

Design 5

Create a PhaseTransitionLogic class that holds a TreeMap<Integer, TransitionRule>, where each rule is assigned a unique priority. Each turn, it checks the rules in ascending priority order and returns the first phase index that differs from the current one. If no rule matches, the current phase is retained.

Pros	Cons
Allows rules to be checked in priority order, supporting structured fallback and emergency logic	Requires unique priority values; incorrect assignment can cause unexpected rule ordering
Keeps TimePhaseManager focused by moving transition decisions into a separate class	Rule ordering must be planned carefully to avoid logic conflicts
Easy to extend by adding new rules with new priorities, without modifying existing logic	

Future extensibility:
If a new rule (e.g. SkipIfConditionRule) is introduced, it can be added to the TreeMap with an appropriate priority without modifying any existing rules or logic.

Design 6

Create a WorldTrigger class to represent a special NPC that handles phase updates each turn. It is immune to attacks and applies the PhaseTickEffect using the TimePhaseManager.

Pros	Cons
Separates phase update logic from other actors and the engine, keeping responsibilities clear and isolated	Only exists to trigger one effect per turn, which may feel unnecessary in simpler systems

Future extensibility:
If new ticking effects are introduced in the future (eg. WeatherTickEffect), they can be attached to WorldTrigger, since it serves as a passive actor that holds global status effects and triggers them each turn using the engine's tick system.

Design 7

Create a PhaseTickEffect class that extends from the engine's StatusEffect class to apply the phase update logic each turn. This status effect is attached to WorldTrigger, which triggers it every turn through the engine's tick system.

Pros	Cons
Encapsulates phase update logic in a reusable and testable effect class	Requires an actor (e.g.WorldTrigger) to hold this effect for it to trigger
Reuses the engine's tick system to perform updates without modifying engine code	
Makes the phase update logic reusable across different actors or systems if needed	

Design 8

Create a CrimsonWell class that extends from the engine's Ground class to represent a temporary crimson tile. If the same actor stands on it for two consecutive turns, it rewards a random potion and then reverts back to the original ground it replaced. It also reverts early if it gains the CRIMSON_END capability.

Pros	Cons
Tracks standing behaviour and handles both reward and revert logic internally, avoiding	

the need for instance of checks	

Design 9

Create a Potion class that extends the engine's Item class and implements the Consumable interface. It stores a SideEffect and applies it when consumed. It can be consumed by actors that have the CONSUME ability.

Pros	Cons
Accepts any SideEffect, so new potions can be added without creating separate subclasses	
Implements Consumable, allowing ConsumeAction to work polymorphically across different items	
Keeps the consume logic inside the item, so actors don’t need to handle effect details	

Future extensibility:
If new consumable potions like FocusElixir is added, they can reuse the same Potion class by supplying different SideEffect instances, avoiding the need to create new subclasses or change existing logic.

Design 10

Added new Status and Ability enum values (CRIMSON, CRIMSON_END, CANNOT_BE_ATTACKED, SELL_ABILITY, DUSKFALL_PHASE, DAWNVEIL_PHASE, CRIMSON_ECLIPSE_PHASE) to support Crimson tile behaviour, WorldTrigger immunity, merchant sleep logic and phase identification.

Pros	Cons
Avoids using instanceof by letting classes check capabilities directly using hasCapability(...)	Logic may fail silently if a required capability is missing

Design 11

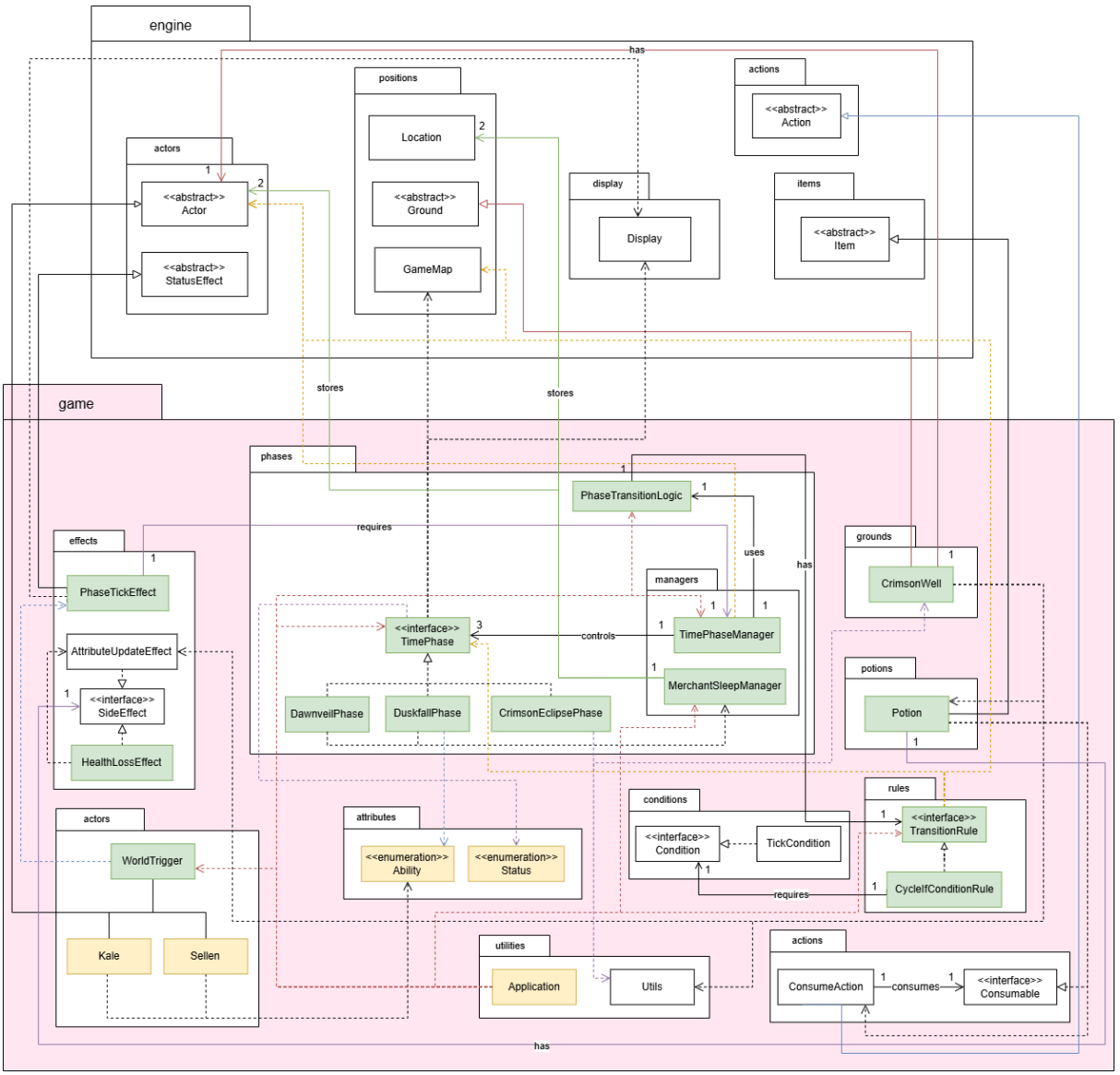
Created a HealthLossEffect class which implements SideEffect and handles health reduction followed by an unconscious check. It wraps AttributeUpdateEffect and centralises death logic for reuse across potions, traps and other damage sources.

Pros	Cons
Separate death logic from attribute modification logic	May cause reliance on correct usage
New health-related effects (eg. traps, poison) can reuse this class without changes	
Can be used anywhere a SideEffect is expected	

Future extensibility:
If in the future a Poison class is introduced that causes health damage over time, it can reuse HealthLossEffect to apply damage and trigger unconsciousness consistently, without needing to duplicate logic.

Future Improvements

- If in the future more complex transitions are required (e.g., jumping directly to BloodmoonPhase when an actor is near Blight tiles), a new helper class like **PhaseRegistry** could be introduced. This registry would allow mapping from PhaseName (enum) to its index in the phase list, making it easier for rules to reference phases by name instead of hardcoded indices. This improves maintainability and reduces the risk of index-related bugs.



The UML above shows the final design for REQ3. It introduces multiple components to support phase management, CrimsonWell behaviour and potion item. These components include a TimePhase interface (Design 1 Approach 1), a MerchantSleepManager (Design 2 Approach 1), a TimePhaseManager (Design 3), a TransitionRule interface (Design 4), a PhaseTransitionLogic class (Design 5), a WorldTrigger NPC (Design 6), a PhaseTickEffect status effect (Design 7), a CrimsonWell ground class (Design 8), a Potion item (Design 9), additional values in the Status and Ability enums (Design 10) and a HealthLossEffect class (Design 11).

TimePhase interface allows each phase (eg. DawnveilPhase, DuskfallPhase and CrimsonEclipsePhase) to define its own environmental effects using applyPhaseEffects() and expose its name. This satisfies **Single Responsibility Principle (SRP)** as each class represents a single phase and only handles its own effects. New phases can be introduced by implementing this interface without modifying existing code, following **Open-Closed Principle (OCP)**. TimePhaseManager interacts with all phases polymorphically through the

TimePhase interface, ensuring **Liskov Substitution Principle (LSP)** is respected. Only essential methods are included in the interface, satisfying **Interface Segregation Principle (ISP)** by keeping the contract minimal.

MerchantSleepManager temporarily removes and stores merchant actors during the Duskfall phase and re-adds them during Dawnveil. This keeps sleep logic encapsulated and avoids scattering it across phase classes, satisfying **SRP**. When a new phase needs merchant sleep logic, it can reuse this manager without rewriting logic, following **DRY**. By centralising the logic for tracking merchant locations, it avoids duplicating the same steps across multiple phases, reducing **connascence of algorithm**. Since future phases can use the same manager without changing its internals, it supports **OCP**.

TimePhaseManager follows **SRP** because it only tracks the current phase and applies its effects; it does not decide when to transition. That decision is delegated to PhaseTransitionLogic, which keeps responsibilities cleanly separated. **OCP** is achieved because new phases can be introduced by implementing the TimePhase interface and adding them to the phase list without modifying the manager itself. Similarly, new transition behaviours can be supported by injecting additional rules into PhaseTransitionLogic. The design also follows **Dependency Injection Principle (DIP)**, as the manager interacts with the TimePhase abstraction instead of concrete classes like DawnveilPhase. This allows different phase types to be swapped in easily. It respects **LSP** because all phase classes implement the same interface methods (applyPhaseEffects, getPhaseName), so they can be used interchangeably. By injecting rules and phases, the design reduces tight coupling and **connascence of algorithm**, keeping each part of the system independent and focused on one task.

The TransitionRule interface defines only the getNextIndex method, achieving **ISP** by exposing the minimal contract needed for transition decisions. This keeps rule classes lightweight and focused. Concrete rules like CycleIfConditionRule follow **SRP**, as they implement just one rule and rely on injected parameters (phase list, actor, map) to evaluate transitions without storing state. Both TimePhaseManager and PhaseTransitionLogic depend only on the TransitionRule interface, not specific implementations, satisfying **DIP** and making the system easy to extend. **OCP** is also achieved, since new rule types can be introduced by adding new classes that implement TransitionRule without modifying existing code. Each rule is evaluated independently, and no rule relies on another, which lowers **connascence of algorithm**. This makes the rules easy to maintain, swap, or reorder without affecting others, keeping logic modular and promoting flexible extension of the transition system.

PhaseTransitionLogic uses a TreeMap to store TransitionRule objects, each with a unique priority. It checks rules in order of ascending priority and stops once it finds one that returns a new phase index. This satisfies **SRP** because the class only decides when a transition should occur, without managing phase state or applying effects. **DIP** is achieved by relying only on the TransitionRule interface, making it easy to swap or extend logic without needing specific

rule types. Since new rules can be added to the TreeMap without changing the class itself, **OCP** is also fulfilled. Rules are evaluated in a defined order, and only one applies per turn, which ensures clear fallback behaviour and low **connascence of execution**. All rule logic is injected, keeping dependencies visible and testable. Because every rule is treated the same regardless of its type, and evaluated via the interface, the class also satisfies **LSP**.

WorldTrigger is a passive NPC that holds PhaseTickEffect and triggers phase updates every turn. This class exists solely to encapsulate ticking logic away from active players or other actors, supporting **SRP**. By isolating the responsibility of ticking from the player or engine loop, it avoids coupling unrelated systems and keeps the update logic modular. It uses capabilities to mark itself as unattackable (CANNOT_BE_ATTACKED), avoiding hardcoded exceptions and reducing **connascence of meaning**.

PhaseTickEffect extends StatusEffect and encapsulates the logic for applying phase updates each turn, achieving **SRP** by keeping ticking logic separate from actors or the engine. It delegates to TimePhaseManager and can be attached to any actor (e.g.WorldTrigger), so phase updates trigger automatically without changing the engine or other actors. This supports **OCP** as new ticking triggers can be added just by attaching the effect. Since the logic is centralised in one class, it avoids repeating update calls across actors, satisfying **DRY**.

The CrimsonWell class adheres to **SRP** by encapsulating all responsibilities related to crimson tile behavior, including tracking which actor stands on it, handling potion rewards, and reverting to the original ground. This keeps the logic cohesive and focused. It also follows **OCP** by using the CRIMSON_END capability to support early reversion without modifying internal logic, allowing external systems to control the tile's lifecycle declaratively. Since it extends the engine's Ground class and maintains expected ground behavior, it satisfies **LSP**, ensuring it can be used wherever a Ground is expected.

The Potion class follows **SRP** by focusing solely on how the item is consumed, while the SideEffect abstraction encapsulates what the effect actually does, keeping responsibilities clearly separated. It satisfies **OCP**, as new potion types can be introduced by injecting different SideEffect instances without modifying the Potion class. By implementing the Consumable interface, it achieves **LSP** by allowing potions to be passed into ConsumeAction or handled by any logic expecting a Consumable, without knowing the concrete class. The design also upholds **DIP** by depending on abstractions (Consumable, SideEffect) rather than specific implementations, enabling flexible reuse and reducing tight coupling.

The new Status and Ability enum values (CRIMSON, CRIMSON_END, CANNOT_BE_ATTACKED, SELL_ABILITY, DUSKFALL_PHASE, DAWNVEIL_PHASE, CRIMSON_ECLIPSE_PHASE) support Crimson tile behaviour, WorldTrigger immunity, merchant sleep logic and time phase tracking. This design supports the **OCP** by allowing new behaviours to be added without modifying existing classes, as components can check capabilities using hasCapability(...) instead of relying on instanceof,

making the system more extensible and reducing coupling. However, this introduces a **connascence of meaning** between the component assigning the capability and the one checking it. If a required capability is not properly assigned, the logic may fail silently, making the connection fragile and dependent on shared assumptions.

The HealthLossEffect class encapsulates damage and death handling by reducing an actor's health and checking if they should fall unconscious. This satisfies **SRP** by separating health-related consequences from attribute modification logic. New effects that cause health loss can reuse this class without requiring changes, aligning with **OCP**. Since HealthLossEffect implements the SideEffect interface, it can be used polymorphically wherever a SideEffect is required, ensuring the **LSP** is respected. It also adheres to **DIP** as both the potion system and environmental effects depend on the high-level SideEffect abstraction rather than concrete effect implementations.